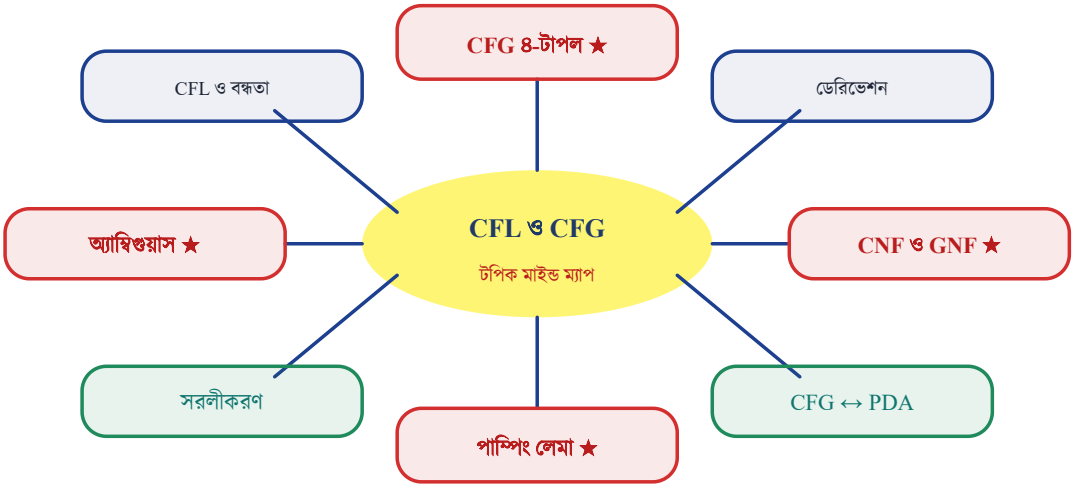
কনটেক্সট-ফ্রি ভাষা ও CFG

CFL ও বন্ধতা · CFG 8-টাপল · ডেরিভেশন · অ্যাম্বিগুয়াস · CNF/GNF · পাম্পিং লেমা · PDA: ১০ পাতায়



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = পদ্ধতি / প্রয়োগ

সিলেবাসের যে অংশ

"Language theory; Finite automata: …pushdown automata; **Context free languages**; **Context free grammars**; Turing Machines: basic machines, configuration, computing with Turing machines." এই নোটে আছে **কনটেক্সট-ফ্রি ভাষা ও গ্রামার** নিজেদের বিস্তার; যে PDA তা চেনে সে অটোমাটা নোটে, টিউরিং মেশিন পরের নোটে।

হাই-ইল্ড তালিকা

① CFG = 8-টাপল (V, Σ, P, S)
② বাম পাশে একটিমাত্র ভেরিয়েবল = CF
③ S → aSb | ε দেয় $a^n b^n$, যেখানে **n ≥ 0**
④ CNF = A → BC বা A → a; GNF = A → aα
⑤ CFL বদ্ধ ∪, concat, * — **নয়** ∩ · তবে CFL ∩ regular = CFL

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = মুখস্থ রাখার তথ্য · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

সূচিপত্র

- ✓ কনটেক্সট-ফ্রি ভাষা ..... পৃ. ২
- ✓ CFG 8-টাপল ..... পৃ. ৩
- ✓ ডেরিভেশন, পার্স ট্রি, অ্যাম্বিগুয়াস ..... পৃ. ৪
- ✓ কাজের উদাহরণ: সুষম বন্ধনী ..... পৃ. ৫
- ✓ সরলীকরণ: CNF ও GNF ..... পৃ. ৬
- ✓ পাম্পিং লেমা, CYK, স্টেপ গণনা ..... পৃ. ৭
- ✓ CFG ↔ PDA ও প্রয়োগ ..... পৃ. ৮
- ✓ পরীক্ষার আগের রাতের পাতা ..... পৃ. ৯
- ✓ অনুশীলন MCQ + লিখিত ..... পৃ. ১০

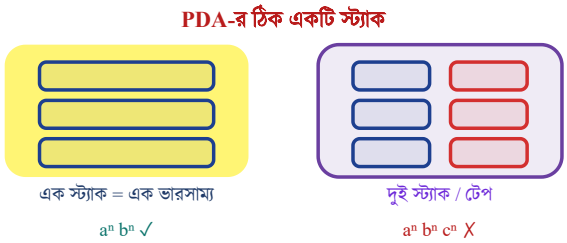
১ কনটেক্সট-ফ্রি ভাষা

সংজ্ঞা: কোনো ভাষাকে **কনটেক্সট-ফ্রি** (context-free) বলা হয় যখন একটি **কনটেক্সট-ফ্রি গ্রামার** তা **তৈরি** করতে পারে এবং একটি **পাশডাউন অটোমাটন** তা চিনতে পারে — চমস্কির **টাইপ ২**। একটি স্ট্যাক একসময়ে মাত্র একটি নেস্টেড কাঠামো ধরে রাখতে পারে, তার বেশি নয়।

কনটেক্সট-ফ্রি ভাষা	ভাষাটি কী বলে	গ্রামার
$\{a^n\, b^n \mid n \geq 0\}$	যত a তত b	$S \to aSb \mid \varepsilon$
$\{w\, w^R \mid w \in \{a,b\}^*\}$	{a,b}-এর সব প্যালিন্ড্রোম	$S \to aSa \mid bSb \mid a \mid b \mid \varepsilon$
$\{a^n\, b^m\, c^m \mid n,m \geq 0\}$	প্রথম ব্লক ফ্রি, পরের দুটি জোড়	$S \to AB \cdot A \to aA\|\varepsilon \cdot B \to bBc\|\varepsilon$
$\{ (^n)^n \mid n \geq 0 \}$	সুষম বন্ধনী	$S \to (S) \mid SS \mid \varepsilon$
$\{a^n\, b^n\, c^n\}$	**কনটেক্সট-ফ্রি নয়**	দুটি স্ট্যাক লাগে

বন্ধতার (closure) পূর্ণ ছক

ভাষার উপর ক্রিয়া	CFL বদ্ধ?
ইউনিয়ন	**হ্যাঁ**
কনক্যাটেনেশন	**হ্যাঁ**
ক্লিন স্টার	**হ্যাঁ**
কমপ্লিমেন্ট	**না**
ইন্টারসেকশন	**না**
রেগুলার ভাষার সাথে ইন্টারসেকশন	**হ্যাঁ**



এক স্ট্যাক একটি ভারসাম্য গোনে, তাই $a^nb^nc^n$ নাগালের বাইরে

কারণ কী: ইউনিয়ন: নতুন শুরু প্রতীক দিয়ে যেকোনো একটি গ্রামার বেছে নাও · কনক্যাটেনেশন: $S_1 \to S_2$, তারপর L_2-এর নিয়ম · স্টার: $S \to SS \mid \varepsilon$, তারপর L-এর নিয়ম · কমপ্লিমেন্ট: PDA-র গ্রহণকারী অবস্থা উল্টালে কোনো CFG পাওয়া যায় না · রেগুলারের সাথে ইন্টারসেকশন: আগে PDA, তার উপর ফিনিট অটোমাটন।

পরীক্ষার ফাঁদ: "ইন্টারসেকশনে বদ্ধ নয়" — এর প্রমাণও মনে রাখতে হয়: $L_1 = \{a^n\, b^n\, c^m\}$ ও $L_2 = \{a^m\, b^n\, c^n\}$ দুটিই কনটেক্সট-ফ্রি, কিন্তু $L_1 \cap L_2 = \{a^n\, b^n\, c^n\}$ **নয়**। তাই উত্তর হলো **সাধারণভাবে নয়** — তবু CFL ∩ **রেগুলার** সবসময় CFL।

মনে রাখার কৌশল: রেগুলার = ∪ ∩ ¬ * · CFL = ∪ · *। রেগুলার ভাষার যে একমাত্র বন্ধতা CFL-এর নেই, তা হলো **ইন্টারসেকশন**।

সমাধানকৃত উদাহরণ ✍️ $\{a^n\ b^m\ c^m \mid n,m \geq 0\}$-এর গ্রামার:

```
V = {S, A, B},  Sigma = {a, b, c},  S = S
P = { S -> A B ,    A -> a A | e ,    B -> b B c | e }

derivation of "aabbcc":
S => AB => aAB => aaAB => aaB => aabBc => aabbBcc => aabbcc
```

A নিয়ম **a**-ব্লককে মুক্ত রাখে, **B** নিয়ম **b** ও **c** জোড়া লাগায় — তাই n, m আলাদা হতে পারে।

২ কনটেক্সট-ফ্রি গ্রামার: ৪-টাপল

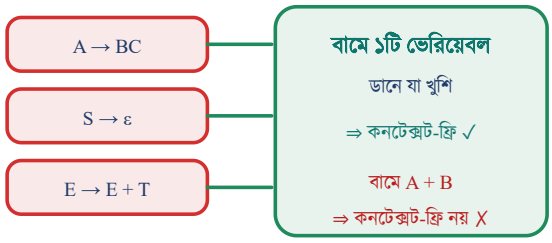
সংজ্ঞা: **কনটেক্সট-ফ্রি গ্রামার** (context-free grammar) হলো $G = (\mathbf{V}, \mathbf{\Sigma}, \mathbf{P}, \mathbf{S})$ — ভেরিয়েবল, টার্মিনাল, প্রোডাকশন ও শুরু প্রতীক। এটি বারবার একটি ভেরিয়েবলকে তার কোনো প্রোডাকশনের ডান পাশ দিয়ে বদলে লিখে $L(G)$ তৈরি করে। এটি কনটেক্সট-ফ্রি ঠিক তখনই, যখন **বাম পাশে ঠিক একটিমাত্র ভেরিয়েবল** — পাশে আর কিছু নেই।

অংশ	কী	উদাহরণ G-তে
V	**ভেরিয়েবলের** সসীম সেট; $S \in V$	$V = \{S, A, B\}$
Σ	**টার্মিনালের** সসীম সেট; $V \cap \Sigma = \emptyset$	$\Sigma = \{a, b\}$
P	**প্রোডাকশনের** সেট $A \to \alpha$, $\alpha \in (V \cup \Sigma)^*$	$P = \{ S \to aSb, S \to \varepsilon \}$
S	**শুরু প্রতীক**, $S \in V$	$S = S$

$\{a^n\ b^n \mid n \geq 0\}$-এর গ্রামার

```
G = ( {S}, {a, b}, P, S )
P = { S -> a S b ,  S -> e }

leftmost derivation of "aaabbb":
S => aSb => aaSbb => aaaSbbb => aaabbb
```



পুরো সংজ্ঞা এক ছবিতে: বাম পাশে একটি ভেরিয়েবল

প্রতিবার $S \to aSb$ **বামে আরেকটি a, ডানে আরেকটি b জড়ায়**; $S \to \varepsilon$ তা থামিয়ে দেয়। n বার ব্যবহারে পাওয়া যায় a^nb^n, আর $n = 0$ হলে ε — তাই ভাষা $\{a^n\ b^n \mid n \geq 0\}$, $n \geq 1$ **নয়**।

সংকেত	অর্থ
a, b, 0, 1	**টার্মিনাল** — আসল অক্ষর
S, A, T	**ভেরিয়েবল** — বদলানোর জায়গা
$A \to \alpha$	একটি **প্রোডাকশন**
$A \Rightarrow \beta$	এক ধাপ **ডেরিভেশন**
$A \Rightarrow^* w$	শূন্য বা তার বেশি ধাপ, w সব টার্মিনাল
ε	খালি স্ট্রিং, $\lvert\varepsilon\rvert = 0$

মনে রাখার কৌশল: চারটি অক্ষরের ক্রম: **V**ariable, **Σ**ymbol, **P**roduction, **S**tart — "V–Σ–P–S"।

পরীক্ষার ফাঁদ: শর্তটি **বাম** পাশে, ডান পাশে নয়। $A \to aSb$ কনটেক্সট-ফ্রি; $AB \to aSb$ নয়, ডান পাশ যেমনই হোক।

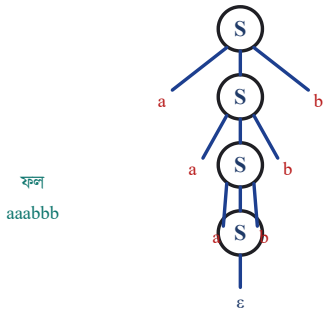
IT link: প্রতিটি প্রোগ্রামিং ভাষার সঙ্গে একটি CFG থাকে **BNF**-তে, যা তার সিনট্যাক্সের **সংজ্ঞাই**; পার্সার জেনারেটর সেই গ্রামার থেকেই যাচাইয়ের টেবিল বানায়।

$V \cap \Sigma = \emptyset$ — কোনো ভেরিয়েবল কখনো টার্মিনালও হয় না, তাই চারটি অংশ কোনোদিন মিশে যায় না

৩ ডেরিভেশন, পার্স ট্রি ও অ্যাম্বিগুয়াস

ডেরিভেশন: S থেকে শুরু করে একবারে একটি ভেরিয়েবল বদলাতে থেকে যাও, যতক্ষণ না কেবল টার্মিনাল থাকে। **লেফটমোস্ট** ডেরিভেশনে সবসময় বাঁকে জেতা ভেরিয়েবলটি বদলায়, **রাইটমোস্ট**-এ ডানে জেতাটি। দুটো একই স্ট্রিংয়ে পৌঁছায় — শুধু **কাজের ক্রম** আলাদা, তাই পার্সার বানানো হয় লেফটমোস্ট।

```
LEFTMOST          RIGHTMOST
S                   S
S => aSb          S => aSb
S => aaSbb        S => aSbb
S => aaaSbbb      S => aaSbb
S => aaabbb       S => aaabbb
```



ফল
aaabbb

S ⇒ aSb ⇒ aaSbb ⇒ aaaSbbb ⇒ aaabbb-এর পার্স ট্রি

পার্স ট্রি = আঁকা ডেরিভেশন: প্রতিটি ভেরিয়েবল একটি নোড; প্রয়োগ প্রতিটি প্রোডাকশন একটি নোড, যার শিশুগুলো ওই নিয়মের ডান পাশের প্রতীকগুলো বাম থেকে ডানে। তাই **পাতা = স্ট্রিং** — বাম থেকে ডানে পড়ে। আর n টি টার্মিনালের ট্রিতে ঠিক $\mathbf{n-1}$টি অভ্যন্তরীণ নোড থাকে।

অ্যাম্বিগুয়াস (Ambiguity): এক স্ট্রিং, দুটি ট্রি

সংজ্ঞা: কোনো গ্রামার **অ্যাম্বিগুয়াস** (ambiguous) যখন তার ভাষার **কোনো** একটি স্ট্রিংয়ের **দুই বা তার বেশি ভিন্ন লেফটমোস্ট ডেরিভেশন** থাকে — সমতুল্যভাবে দুটি পার্স ট্রি। একটিও স্ট্রিং না থাকলে তা **অস্পষ্টতামুক্ত** (unambiguous)। একটি খারাপ স্ট্রিংই যথেষ্ট।

সমাধানকৃত উদাহরণ ✍️ প্রচলিত অ্যাম্বিগুয়াস গ্রামার:

```
E -> E + E | E * E | id

tree 1: (id+id)*id          tree 2: id+(id*id)
        *                           +
       / \                         / \
      +   id                     id   *
     / \                            / \
   id   id                        id  id
```

চারটি টার্মিনাল একই ক্রমে, কিন্তু গ্রুপিং দুটি — তাই **id+id*id** অ্যাম্বিগুয়াস। ঠিক করতে প্রতি অপারেটরের জন্য একটি বাড়তি স্তর দরকার: **E → E + T | T, T → T * F | F, F → id**।

মনে রাখার কৌশল: "অ্যাম্বিগুয়াস = **অ্যাল্টারনেটিভ** ট্রি"। এক স্ট্রিং **দুইভাবে** বানানো গেলে পার্সার বুঝতে পারে না তা মানে কী।

পরীক্ষার ফাঁদ: অ্যাম্বিগুয়াসের সাথে ε-প্রোডাকশন, ভেরিয়েবলের সংখ্যা বা অসীম ভাষার কোনো সম্পর্ক নেই। আর এটি ননডিটারমিনিজমও নয়: PDA প্রতিটি CFL-তেই ননডিটারমিনিস্টিক।

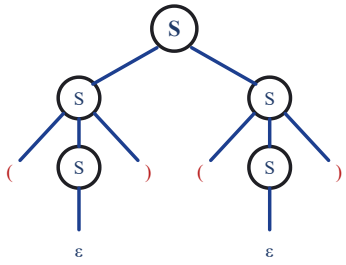
৪ কাজের গ্রামার: সুষম বন্ধনী

সংজ্ঞা: $\Sigma = \{ (,) \}$-এর উপর **সুষম বন্ধনী**র ভাষা হলো $\{ (^n)^n \mid n \geq 0 \}$: প্রতিটি (পরে একটি) দিয়ে বন্ধ, আর নেস্টিং ঠিক থাকে। এটি কনটেক্সট-ফ্রি, আর অটোমাটা নোটের PDA ঠিক (খোলার সময় push,) এলে pop করে এটিই গ্রহণ করে।

গ্রামারটি

```
V = {S},  Sigma = { ( , ) },  S = S
P = { S -> SS ,       দুটি গ্রুপ পাশাপাশি
      S -> (S) ,      এক গ্রুপের ভিতরে আরেকটি
      S -> e }        খালি গ্রুপ

leftmost derivation of "(())()":
S => SS => (S)S => ((S))S => (())S => (())(S) => (())()
```



(())()-এর পার্স ট্রি: দুটি পাশাপাশি গ্রুপ, ভিতরে দুটোই খালি

এটিকে গ্রহণের প্রমাণ হিসেবে পড়ো: প্রতিটি (ঠিক একটি) দিয়ে বন্ধ, আর কোনো) আগে আসে না। উল্টো দিকটাও সত্য — **S → SS** প্রথম গ্রুপ বানিয়ে বাকিটা জুড়ে দেয়, **S → (S)** একটি নেস্টিং স্তর সামলায়।

প্রোডাকশন	এটি কোন ভাবনা ধরে	যা শেষ করতে পারে
S → ε	বেস কেস: খালি গ্রুপ	ε, আর কিছু নয়
S → (S)	**একটি** গ্রুপকে এক জোড়ায় মোড়ানো	( ), (( )), ((())) …
S → SS	দুটি সুষম গ্রুপ জোড়া লাগানো	()(), (())(), (()()) …

মনে রাখার কৌশল: SS = **পাশাপাশি**, (S) = **ভিতরে**, ε = **কিছু নয়**। একই তিন-নিয়মের আকারেই সুষম {}, সুষম [ ], আর তৃতীয় জোড়া যোগ করলে সুষম { } [ ] তৈরি হয়।

IT link: এডিটরের ব্র্যাকেট ম্যাচার একটি PDA: খোলার প্রতীকে push, মিলের প্রতীকে pop, খালি স্ট্যাক থেকে pop নয়। কম্পাইলারে একই গ্রামারই পার্সারের প্রোডাকশন নিয়ম হয়ে যায়।

পরীক্ষার ফাঁদ: লিখবে **S → SS**, কখনো **S → SS | SS** নয়। বিকল্পগুলোর ক্রম অর্থহীন — CFG-র প্রোডাকশন একটি **সেট**, প্রোগ্রাম নয়: যেকোনো নিয়ম যেকোনো সময়ে চলতে পারে। $\{a^n b^n c^n\}$ এভাবে বানানো যায় না — এক স্ট্যাক দিয়ে দ্বিতীয় গণনা চলতেই একটি বন্ধনী বন্ধ করা যায় না।

একই আকারে আর যা বানানো যায়: যেকোনো k ধরনের জোড়া বন্ধনীর সুষম স্ট্রিং (Dyck ভাষা), সুষম XML/HTML ট্যাগ-নেস্টিং, এবং আধা-প্রোগ্রামের **E → E + E | E * E | id** — প্রতিটির জন্য **প্রতি স্তরে একটি ভেরিয়েবল** যথেষ্ট।

```
input:  (  (  )  )  (  )
stack:  Z0 -> Z0 -> (Z0 -> ((Z0 -> (Z0 -> Z0 -> (Z0 -> Z0   (accept)
```

৫ CFG সরলীকরণ: CNF ও GNF

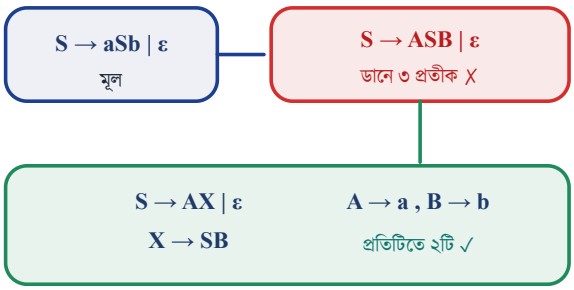
১. **অব্যবহৃত প্রতীক সরাও:** আগে **নন-জেনারেটিং** ভেরিয়েবল, তারপর **অনপ্রাপ্য** ভেরিয়েবল।

২. **ε-প্রোডাকশন A → ε সরাও:** সব **নালেবল** ভেরিয়েবল খুঁজে তার ε-নিয়ম বাদ দাও।

৩. **ইউনিট প্রোডাকশন A → B সরাও** সেই জায়গায় B-র নিজের নিয়ম বসিয়ে (বারবার — চেইন ঘটে)।

৪. **চমস্কি নরমাল ফর্মে** রূপান্তর — CYK ও পার্সার জেনারেটর এটাই চায়।

৫. **গ্রেবাখ নরমাল ফর্মে** রূপান্তর, যেখানে প্রতিটি নিয়ম টার্মিনাল দিয়ে শুরু হয়।



CNF-তে কেবল A → BC বা A → a

তাড়াহুড়ো করে CNF রূপান্তর কোথায় ভুল হয়, আর তার সমাধান

নরমাল ফর্ম	প্রতিটি প্রোডাকশন	যা দেয়
CNF	**A → BC** বা **A → a**; S → ε কেবল ε ∈ L হলে	CYK, yacc/bison
GNF	**A → aα**	এক প্রতীকেই পূর্বানুমান
রাইট-লিনিয়ার	A → aB	একটি **রেগুলার** টাইপ ৩ গ্রামার

সমাধানকৃত রূপান্তর ✍️ S → aSb | ε থেকে CNF:

```
original   S -> a S b | e
step 1     বাঁ টার্মিনালটি বাঁধো, তারপর ডানেরটি:  S -> A S B
step 2     ডানে এখনো ৩ প্রতীক, তাই পুচুরটি ভাঙো:
           S -> A X   ,  X -> S B   ,  A -> a   ,  B -> b
result     S -> A X | e ,  X -> S B ,  A -> a ,  B -> b
```

S → ε কেন টেকে যায়: CNF-তে এটি তখনই বৈধ যখন ε ∈ L **এবং** S কোনো ডান পাশে নেই। এখানে S **আছে**, **X → SB**-এর ভিতরে। তাই নিখুঁত রূপে একটি নতুন শুরু প্রতীক **S_0 → S** যোগ করতে হয়, বাকি চারটি নিয়ম একই থাকে।

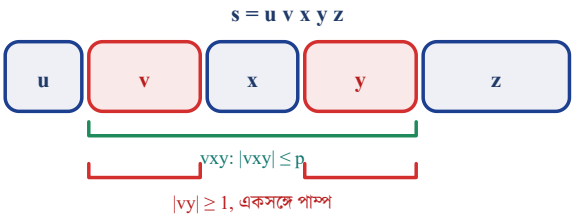
পরীক্ষার ফাঁদ: $S \to ASB$ CNF **নয়** — **ঠিক ২ ভেরিয়েবল বা ১ টার্মিনাল**, তিন প্রতীক কখনো নয়। আর $A \to a\alpha$ হলো **গ্রেবাখ**, CNF নয়।

মনে রাখার কৌশল: **CNF = দুই বা এক, GNF = টার্মিনাল দিয়ে শুরু।** ডান পাশের প্রতীক গুনো: ২ বা ১ ⇒ CNF; "a তারপর ভেরিয়েবল" ⇒ GNF; "aB" ⇒ রেগুলার।

৬ CFL-এর পাম্পিং লেমা, CYK ও স্টেপ গণনা

কনটেক্সট-ফ্রি ভাষার পাম্পিং লেমা: L কনটেক্সট-ফ্রি হলে এমন দৈর্ঘ্য **p** আছে যাতে |s| ≥ p এমন প্রতিটি s ∈ L-কে **s = uvxyz** করে ভাঙা যায়, যেখানে **|vxy| ≤ p**, **|vy| ≥ 1** এবং **প্রতিটি i ≥ 0-এর জন্য $uv^ixy^iz \in L$**। মানে, ছোট একটা জানালার ভিতরে এমন একটি ভারসাম্যপূর্ণ অংশ আছে যা যতবার খুশি পুনরাবৃত্তি করা যায়। রেগুলার সংস্করণের মতো এটিও **খণ্ডনের** হাতিয়ার।

শর্ত	কী বলে
$\|vxy\| \le p$	পাম্পযোগ্য জানালাটি **ছোট**
$\|vy\| \ge 1$	v ও y একসঙ্গে **খালি নয়**
$uv^ixy^iz \in L, \forall i \ge 0$	v ও y **একসঙ্গে** পুনরাবৃত্তি হয়



CFL-এর পাম্পিং লেমার পাঁচ-খণ্ডের ভাঙা

পরীক্ষার ফাঁদ: রেগুলার লেমা পাম্প করে xy^nz; CFL লেমা পাম্প করে uv^ixy^iz — **দুটি** সূচক, এবং সেগুলো **সমান**। $u(vxy)^nz$ লেখা ভুল। আর লেমা মেনে চললেই কোনো ভাষা কনটেক্সট-ফ্রি **প্রমাণ** হয় না।

সমাধানকৃত খণ্ডন ✍️ $\{a^n b^n c^n \mid n \ge 0\}$ কনটেক্সট-ফ্রি নয়:

পাম্পিং দৈর্ঘ্য p নাও আর $s = a^pb^pc^p$ নাও। |vxy| ≤ p ও |vy| ≥ 1 শর্তে যেকোনো ভাঙায় y পুরোপুরি **a-ব্লকের** ভিতরে ($y = a^k, k \ge 1$) অথবা পুরোপুরি **b-ব্লকের** ভিতরে ($y = b^k$)। একা y পাম্প করলে মিল ভেঙে যায়, আর v ও y **একসঙ্গে** পাম্প করলেও তা ফেরত আসে না, কারণ তারা ভিন্ন ব্লকে। শর্ত ভাঙে ⇒ **L কনটেক্সট-ফ্রি নয়**। এক বাক্যে: এখানে তিনটি একসঙ্গে গণনা দরকার, আর এক স্ট্যাক একটিই ধরে।

CYK ও স্টেপ গণনা: **CYK** একটি **নির্দিষ্ট CNF** গ্রামারের জন্য **$O(n^3)$** সময়ে সিদ্ধান্ত নেয় (সাধারণত $O(n^3 \cdot |G|)$): $O(n^2)$ উপস্ট্রিংয়ের উপর নিচ থেকে ওপরের টেবিল, প্রতিটির জন্য $O(n)$ স্প্লিট পয়েন্ট। আর CNF-তে দৈর্ঘ্য **$n \geq 1$** স্ট্রিং ডেরিভ করতে ঠিক **$2n - 1$** ধাপ লাগে — একটি ভেরিয়েবলকে n টি করতে **$n - 1$** ধাপ $A \to BC$, তারপর প্রতিটির জন্য একটি করে **n** ধাপ $A \to a$।

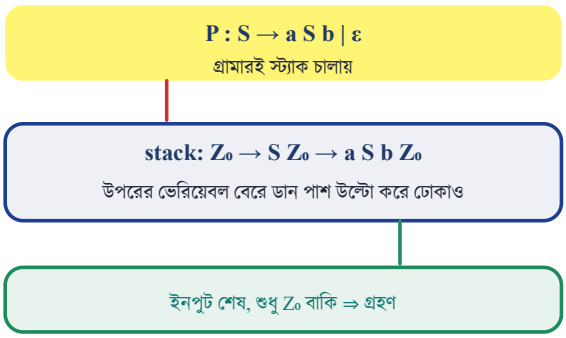
মনে রাখার কৌশল: **"n − 1 ভাগ, তারপর n টার্মিনাল = মোট 2n − 1"** — আর **CYK ঘনঘন (cubic)**: উপস্ট্রিং, স্প্লিট পয়েন্ট ও নিয়মের তিনটি লুপ।

> রেগুলার লেমার মতো এখানেও কোনো উপায়ে কনটেক্সট-ফ্রি প্রমাণ করা যায় না — শুধু খণ্ডন করা যায়

৭ CFG ↔ PDA এবং CFG-র প্রয়োগ

সমতার উপপাদ্য (হাই-ইল্ড): **কনটেক্সট-ফ্রি ⇔ কোনো PDA তা চিনে**। "যদি" দিকটা সহজ — গ্রামারটিকে একটি লেফটমোস্ট ডেরিভেশন হিসেবে পড়ো। "কেবল যদি" দিকটাই চমকপ্রদ: PDA-র **অবস্থাগুলোই ভেরিয়েবল হয়ে যায়**, তাই মেশিনটি নিজেই নিজের গ্রামার লিখে ফেলে।

১. **শুরু প্রতীক S** স্ট্যাকে, বটম মার্কার Z_0-এর উপরে push করো।
২. স্ট্যাকের **উপরে ভেরিয়েবল** A থাকলে এবং P-তে $A \to \alpha$ থাকলে, A বের করে α **বিপরীত** ক্রমে push করো, যাতে α-র বামতম প্রতীক আগে বেরোয়।
৩. স্ট্যাকের **উপরে টার্মিনাল** a থাকলে পরের ইনপুটের সাথে মেলাও: সমান হলে pop ও ইনপুট এগোও, নয়তো প্রত্যাখ্যান।
৪. ইনপুট শেষ **এবং** স্ট্যাকে শুধু Z_0 আছে — তবেই গ্রহণ।
৫. উল্টো দিক: প্রতিটি অবস্থা-জোড়া p, q ও স্ট্যাক প্রতীক X-এর জন্য একটি ভেরিয়েবল **[pXq]**, প্রতিটি ট্রানজিশনে **[pXq] → a**, আর প্রতিটি গ্রহণকারী অবস্থা q-এর জন্য **$S_0 \to [q_0\ Z_0\ q]$**।



CFG → PDA: গ্রামারটিই স্ট্যাকের হাত ধরে

প্রয়োগ	CFG কীভাবে ব্যবহৃত হয়
প্রোগ্রামিং ভাষা	C, Java, Pascal-এর সিনট্যাক্স BNF/EBNF-তে সংজ্ঞায়িত — গ্রামারটিই **সংজ্ঞা**
কম্পাইলার	**পার্সার জেনারেটর** (yacc, bison, ANTLR) গ্রামার পড়ে টেবিল বানায়
মার্কআপ ভাষা	XML / HTML-এর নেস্টিং একটি CFG; ভ্যালিডেটর হলো ট্যাগ-স্ট্রিমের উপর PDA
প্রাকৃতিক ভাষা	বাক্যের কাঠামো একটি গ্রামার; প্রতিটি NLP পাইপলাইনের শুরুতে পার্সিং
এক্সপ্রেশন	precedence climbing = প্রতি অপারেটরের জন্য একটি বাড়তি গ্রামার স্তর
কোয়েরি ভাষা	SQL-এর *SELECT … WHERE …* একটি রিকার্সিভ-ডিসসেন্ট গ্রামার

মনে রাখার কৌশল: **শুরু প্রতীক** push, **স্ট্যাকের উপরেরটি** বদলাও, **খালি স্ট্যাকে** গ্রহণ। উল্টো রূপান্তরের ভেরিয়েবলগুলো **[pXq]** — "p থেকে, X-এর উপর, q-তে"।

পরীক্ষার ফাঁদ: CFG → PDA-তে ডান পাশ **বিপরীত** ক্রমে push হয়: স্ট্যাক last-in-first-out। α লেখার ক্রমে ঢোকালে PDA স্ট্রিংটি **উল্টো** দিকে বানাবে।

IT link: ব্র্যাকেট মিলানো, *if/for* নেস্টিং যাচাই আর JSON ভ্যালিডেশন — সবই পাতা ৫-এর একই PDA, শুধু ইনপুট বদলে।

পরীক্ষায় যখন এই ধরনের প্রশ্ন আসে: "গ্রামার থেকে PDA" চাইলে শুরু প্রতীক push · "PDA থেকে গ্রামার" চাইলে **[pXq]** ভেরিয়েবল · "কোন ভাষা" চাইলে শুধু লিখো **{$a^n b^n$ | n ≥ 0}**-এর মতো একটি সেট।

পরীক্ষার আগের রাতের পাতা

তথ্যপাত:

CFL = CFG বানায়, PDA পড়ে

CFL হলো চমস্কি **টাইপ ২**

CFG = **(V, Σ, P, S)**

CF ⇔ প্রতি বাম পাশে একটি ভেরিয়েবল

$V \cap \Sigma = \emptyset$; $S \in V$; $P \subseteq V \times (V \cup \Sigma)^*$

$A \Rightarrow$ এক ধাপ · $A \Rightarrow^* w \cdot \varepsilon =$ খালি

$S \to aSb \mid \varepsilon \Rightarrow \{a^n b^n \mid n \geq 0\}$

$S \to aSa \mid bSb \mid a \mid b \mid \varepsilon \Rightarrow$ প্যালিন্ড্রোম

$S \to SS \mid (S) \mid \varepsilon \Rightarrow$ সুষম বন্ধনী

লেফটমোস্ট = আগে বাঁকের ভেরিয়েবল

পার্স ট্রির পাতা = স্ট্রিং

অ্যাম্বিগুয়াস ⇔ ≥ ২ লেফটমোস্ট ডেরিভেশন

$E \to E+E \mid E*E \mid id$ অ্যাম্বিগুয়াস

বদ্ধ: ∪, কনক্যাটেনেশন, ক্লিন স্টার

বদ্ধ নয়: ∩ ও কমপ্লিমেন্ট

তবে CFL ∩ **রেগুলার CFL**

$\{a^n b^n c^m\} \cap \{a^m b^n c^n\} = \{a^n b^n c^n\}$ = CF নয়

CNF: $A \to BC$ বা $A \to a$ (+ $S \to \varepsilon$)

GNF: $A \to a\alpha \cdot A \to aB$ = রেগুলার

CFL পাম্পিং: $s = \mathbf{uvxyz}$, $|vxy| \leq p$, $|vy| \geq 1$

$uv^i xy^i z \in L \; \forall \; i \geq 0$ — সমান সূচক

নির্দিষ্ট CNF গ্রামারে CYK: $\mathbf{O(n^3)}$

$n \geq 1$ দৈর্ঘ্যের CNF ডেরিভেশন: $\mathbf{2n - 1}$

CFG ⇔ PDA; PDA → CFG-তে **[pXq]**

MCQ-র জন্য শীর্ষ ১২ এক-লাইনের উত্তর

1. CFG = ৪-টাপল, PDA = ৭-টাপল

7. $A \to a\alpha$ হলো **গ্রেবাখ**, $A \to aB$ রাইট-লিনিয়ার

2. বামে একটি ভেরিয়েবল = কনটেক্সট-ফ্রি

3. S → aSb | ε দেয় **n ≥ 0**, ε-সহ

4. টাইপ ৩ গ্রামার দেয় **রেগুলার** ভাষা

5. CFL বদ্ধ ∪ · * — **নয়** ∩

6. CNF-তে কেবল **A → BC বা A → a**

8. অ্যাম্বিগুয়াস = এক স্ট্রিং, দুটি ডেরিভেশন

9. CFL লেমা **পাঁচ** খণ্ডে ভাঙে: uvxyz

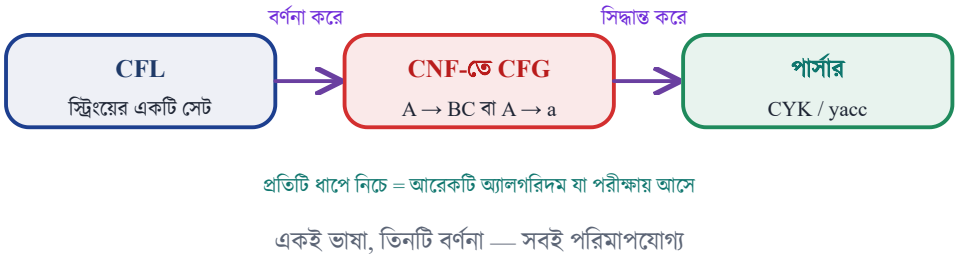
10. CYK ইনপুট দৈর্ঘ্যের **ঘনঘন**

11. CNF-এর স্টেপ গণনা **2n − 1**

12. CFL ∩ রেগুলার = CFL

শীর্ষ ফাঁদ: ডানে তিন প্রতীক মানে **CNF নয়** · "ইন্টারসেকশনে বদ্ধ নয়"-এর সাথে রেগুলার-ভাষার যোগসূত্র দরকার · CFL পাম্পিং লেমায় v ও y একই সূচকে পাম্প হয় · লেমা মেনে চললেই কনটেক্সট-ফ্রি প্রমাণ হয় না · PDA প্রোডাকশনের ডান পাশ **বিপরীত** ক্রমে ঢোকায় · n ≥ 0 ভাষায় ε থাকে।

সব কৌশল একসাথে: "V–Σ–P–S, চারটি অংশ" · "বাম পাশে একটি ভেরিয়েবলই পুরো সংজ্ঞা" · "SS = পাশাপাশি, (S) = ভিতরে, ε = কিছু নয়" · "পাতা বাম থেকে ডানে = স্ট্রিং" · "CNF = দুই বা এক, GNF = টার্মিনাল দিয়ে শুরু" · "পাঁচ খণ্ড, দুটি সূচক" · "n − 1 ভাগ + n টার্মিনাল = 2n − 1"।



✓ অনুশীলন MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. একটি CFG কতগুলো অংশের টাপল?

ক) 3 খ) 4

গ) 5 ঘ) 7

২. CNF-তে প্রতিটি প্রোডাকশন কোন আকারের?

ক) A → aα খ) A → aB

গ) A → BC | A → a ঘ) যেকোনো α, |α| ≤ 2

৩. টাইপ ৩ গ্রামার কোন ভাষা তৈরি করে?

ক) কনটেক্সট-ফ্রি খ) কনটেক্সট-সেনসিটিভ

গ) রেগুলার ঘ) রি. এনিউমারেবল

৪. CFL কোন ক্রিয়ার অধীনে **বদ্ধ নয়**?

ক) ইউনিয়ন খ) কনক্যাটেনেশন

গ) ক্লিন স্টার ঘ) ইন্টারসেকশন

৫. S → aSb | ε কোন ভাষা তৈরি করে?

ক) $\{a^n b^n \mid n \geq 1\}$ খ) $\{a^n b^n \mid n \geq 0\}$

গ) (ab)* ঘ) a*b*

৬. গ্রামার কখন অ্যাম্বিগুয়াস?

ক) ২+ ট্রি, ১ স্ট্রিং খ) ε-নিয়ম আছে

গ) অসীম ভাষা ঘ) >১ ভেরিয়েবল

৭. CYK সদস্যপতা কত সময়ে নির্ণয় করে?

ক) O(n) খ) O(n log n)

গ) $O(n^2)$ ঘ) $O(n^3)$

৮. CNF-তে দৈর্ঘ্য n ≥ 1 ডেরিভ করতে কত ধাপ?

ক) n ধাপ খ) 2n − 1 ধাপ

গ) 2n ধাপ ঘ) n^2 ধাপ

৯. $\{a^n b^n c^n \mid n \geq 1\}$ কোনটি?

ক) রেগুলার খ) কনটেক্সট-ফ্রি

গ) দুটোই নয় ঘ) দুটোই

১০. CFL ∩ একটি **রেগুলার** ভাষা কী?

ক) সবসময় CFL খ) নিশ্চয় নয়

গ) কখনো CFL নয় ঘ) রেগুলার ভাষা

উত্তরমালা → ১-খ · ২-গ · ৩-গ · ৪-ঘ · ৫-খ · ৬-ক · ৭-ঘ · ৮-খ · ৯-গ · ১০-ক

লিখিত প্রশ্ন: {a, b}-এর উপর সব প্যালিন্ড্রোমের ভাষার জন্য একটি CFG লেখ; *abba* ও *aba*-র লেফটমোস্ট ডেরিভেশন দেখাও; বলো কেন এটি কনটেক্সট-ফ্রি কিন্তু রেগুলার নয়। ১০ নম্বর

মডেল উত্তর ✍️:

১ **গ্রামার** 3 — G = ({S}, {a,b}, P, S), P = { **S → aSa | bSb | a | b | ε** }: দুই প্রান্ত জড়াও, মাঝটা শেষ করো।

২ **ডেরিভেশন** 3 — abba: S ⇒ aSa ⇒ abSba ⇒ **abba** · aba: S ⇒ aSa ⇒ **aba**।

৩ **কেন কনটেক্সট-ফ্রি** 2 — বাম পাশে একটি ভেরিয়েবল; সমতুল্যভাবে একটি NPDA প্রথম অর্ধেক push করে, মাঝ **অনুমান** করে, দ্বিতীয় অর্ধেক pop করে।

৪ **কেন রেগুলার নয়** 2 — পাম্পিং দৈর্ঘ্য p নিয়ে w = $a^p\ b\ a^p$ নিলে y = a^p-ব্লকের ভেতরে পড়ে, তাই xy^2z প্যালিন্ড্রোম নয়।

মুখস্থের ছক: প্যালিন্ড্রোম **S → aSa | bSb | a | b | ε** · বন্ধনী **S → SS | (S) | ε** · "দুই প্রান্ত জড়াও, মাঝ শেষ করো"।

পরীক্ষার আগে যাচাই ✓

- ✓ (V, Σ, P, S) মেমরি থেকে লিখতে পারি
- ✓ a^nb^n, প্যালিন্ড্রোম, বন্ধনীর গ্রামার দিতে পারি
- ✓ ছোট গ্রামারকে CNF-তে রূপান্তর করতে পারি

আরও অনুশীলন: $\{a^nb^nc^n\}$-এর পাম্পিং-লেমা প্রমাণ · PDA → CFG · একটি CYK টেবিল